

Automated Partitioning & Channel-Coloring Compiler for Distributed Biocomputing

Sources referenced:

Padmakumar et al (Partitioning of a 2-bit hash function across 66 communicating cells),
Densmore/Voigt CAD research (Droplet Microfluidics Co-Designed with Real-Time CMOS
Luminescence Sensing and Impedance Spectroscopy of 4nL Droplets at a 67mm/s Velocity)

Core Challenge:

Large synthetic logic circuits exceed the capacity of a single cell due to severe cellular resource burden and cross-reactivity, limiting single cells to ~10 gates.

Dividing a circuit (such as the 110-gate MD5 hash circuit) across a consortium of communicating strains requires grouping gates under a maximum capacity constraint (8 gates/cell) and solving an edge-coloring problem via Welsh–Powell node coloring to assign orthogonal chemical communication wires (e.g., OC6, OHC14, pC-HSL, DAPG) without signal crosstalk.

For synthetic biologists who need to construct genetic logic circuits too large for a single bacterial cell, a Automated Partitioning & Channel-Coloring Compiler would be a CAD automation tool that automatically partitions digital logic netlists across minimal bacterial subcircuit strains and assigns orthogonal chemical communication channels without signal crosstalk. Unlike manual circuit layout or traditional electronic CAD tools that fix the chassis count, it dynamically balances strain allocation while enforcing biological per-cell resource burden limits (8 gates/cell) and chemical signaling constraints.

Users:

Primary User (Computational Synthetic Biologist/Postdoc): Designs complex genetic logic networks in Linux/Python CAD environments. They need to compile large Boolean circuits (synthesized via Yosys) into physically buildable DNA constructs while avoiding cellular resource burden and cross-reactivity.

Secondary User/Decider (Bio-foundry Lead/Principal Investigator): Oversees lab research and DNA assembly budgets. They require guaranteed subcircuit isolation, compliance with Cello compilation standards, and minimization of physical strain construction workloads. They prefer something that costs less than it would take to pay their CSB or postdoc to do the work without the tool.

WWWWWH:

- What: A design tool that takes in a digital logic netlist and outputs a list assignment of chemical communication channels without signal cross talk.
- Who: Computational Synthetic Biologist/Postdoc, Lab supervisor, Principal Investigator.
- Why: In my experience, the biology students I talked to do not tend to have much programming experience. Helping them to automate their
- When: Near the beginning of the project, to provide path options for the users to choose between and improve.

- Where: Software that interfaces with databases (Genbank, InterPro, Pfam, etc).
- How: Takes user inputs (logic gate netlist, integer requirements of maximum number of gates per cell, maximum number of strains, maximum number of bacteria), and outputs a list of bacteria strains that could be used along with specifying the connections between them.

Assumptions/Validation table:

<u>Assumption</u>	<u>Validation</u>
Channel Mapping Feasibility: Welsh-Powell node coloring can map arbitrary inter-subcircuit connections across a 110+ gate netlist using orthogonal chemical channels without channel collisions or graph-coloring deadlocks.	Run a standalone Python script to execute Welsh-Powell node coloring on 1000 synthetic random graph partitions of the MD5 circuit.
Strain Count Tractability: Enforcing a strict limit of 8 gates/cell and limiting number of total required physical strains whilst allowing variable cell count for the standard benchmark circuits produced.	Ingest Yosys-synthesized Verilog netlists for 3 benchmark circuits (for example: MD5, 4-bit adder, ALU) into a lightweight greedy partitioning prototype script.
Cello/CAD Interoperability: Exported structural Verilog files specifying sender/receiver signaling devices parse cleanly and compile into DNA sequences using Cello 2.1.	Hand-template 5 structural Verilog subcircuits with sender/receiver definitions and run them through the Cello 2.1 API/CLI.
Optimization Algorithm Value: Advanced optimization heuristics (e.g., Integer Linear Programming or Simulated Annealing) produce significantly fewer strains (15% reduction) than the stochastic greedy baseline (n=1,000\$)	Compare an ILP/Simulated Annealing prototype against the 1,000-iteration greedy baseline on the MD5 core netlist.
User Need & Workflow Fit: Computational synthetic biologists need an automated tool that dynamically solves both gate partitioning and channel coloring simultaneously, rather than relying on manual partitioning or fixed-chassis electronic CAD tools.	Present a 1-page workflow mock-up and 5 user stories to 3 synthetic biology researchers or bio-foundry staff. Suggestions: Prof Rabia Yazicigil, Prof Doug Densmore.

Potential Involvement of LLM:

Turn natural language inputs into account for the requirements that the software uses, or create an option for recalculation using natural language feedback.

Things I would need to learn:

How does Welsh-Powell node colouring work?

How do I use Cello 2.1?

How do I use Yosys?

How do I access the databases required?

How do I get information from the databases mentioned, and are there limits to how much information I can request?

Problems with this project:

Not all databases will be alright with AI crawlers/bots requesting large amounts of information. Is it possible to reduce the number of requests to an amount that the scientist/postdoc user would send themselves, or avoid the usage of said databases altogether? Must prioritise interacting solely through official APIs and not through data crawling.

